

Skill-03 Task-01: Command History

Objective

To implement command history functionality by applying escape sequences, storing commands, navigating previous and next commands, updating the input buffer, and testing command recall.

Features

- Accept keyboard input
- Detect Enter key
- Handle Backspace
- Detect escape sequences
- Store commands in history
- Navigate previous commands using the Up arrow
- Navigate next commands using the Down arrow
- Update the input buffer
- Support multiple commands
- Exit using `exit` or Ctrl+D

Escape Sequences

The terminal sends escape sequences for special keys.

The Up arrow is represented by:

ESC [A

Code-

```
#include <stdio.h> #include <stdlib.h> #include <string.h> #include <unistd.h>
#include <termios.h>
```

```
#define MAX_HISTORY 100 #define BUFFER_SIZE 1024
```

```
char *history[MAX_HISTORY]; int history_count = 0; int history_position = -1;
```

```
void disable_raw_mode(struct termios *original) { tcsetattr(STDIN_FILENO,
TCSAFLUSH, original); }
```

```
void enable_raw_mode(struct termios *original) { struct termios raw;
```

```

tcgetattr(STDIN_FILENO, original);
raw = *original;

raw.c_lflag &= ~(ECHO | ICANON);
tcsetattr(STDIN_FILENO, TCSAFLUSH, &raw);

}

void add_history(const char *command) { if (strlen(command) == 0) return;

if (history_count == MAX_HISTORY)
{
    free(history[0]);

    for (int i = 1; i < MAX_HISTORY; i++)
        history[i - 1] = history[i];

    history_count--;
}

history[history_count] = strdup(command);
history_count++;

}

void clear_line(int length) { for (int i = 0; i < length; i++) write(STDOUT_FILENO, "\b \b",
3); }

void display_history_command(char *buffer, int *length, int position) { if (position >= 0
&& position < history_count) { clear_line(*length);

    strcpy(buffer, history[position]);
    *length = strlen(buffer);

    write(STDOUT_FILENO, buffer, *length);
}

}

int main() { struct termios original; char buffer[BUFFER_SIZE]; int length = 0;

```

```

enable_raw_mode(&original);

printf("Simple Command History\n");
printf("Type commands. Use Up/Down arrows to navigate.\n");
printf("Press Ctrl+D or type exit to quit.\n\n");

while (1)
{
    printf("\nshell> ");
    fflush(stdout);

    length = 0;
    buffer[0] = '\0';
    history_position = history_count;

    while (1)
    {
        char c;

        if (read(STDIN_FILENO, &c, 1) != 1)
            break;

        /* Enter key */
        if (c == '\n' || c == '\r')
        {
            buffer[length] = '\0';
            printf("\n");

            if (strcmp(buffer, "exit") == 0)
            {
                disable_raw_mode(&original);

                for (int i = 0; i < history_count; i++)
                    free(history[i]);

                return 0;
            }

            if (length > 0)
            {
                add_history(buffer);
                printf("Command stored: %s\n", buffer);
            }
        }
    }
}

```

```

    }

    break;
}

/* Ctrl+D */
if (c == 4)
{
    disable_raw_mode(&original);

    for (int i = 0; i < history_count; i++)
        free(history[i]);

    printf("\nExiting...\n");
    return 0;
}

/* Backspace */
if (c == 127 || c == 8)
{
    if (length > 0)
    {
        length--;
        buffer[length] = '\0';
        write(STDOUT_FILENO, "\b \b", 3);
    }

    continue;
}

/* Escape sequence */
if (c == 27)
{
    char seq[2];

    if (read(STDIN_FILENO, &seq[0], 1) != 1)
        continue;

    if (read(STDIN_FILENO, &seq[1], 1) != 1)
        continue;

    /* Up arrow */

```

```

if (seq[0] == '[' && seq[1] == 'A')
{
    if (history_count > 0 && history_position > 0)
    {
        history_position--;
        display_history_command(
            buffer,
            &length,
            history_position
        );
    }

    continue;
}

/* Down arrow */
if (seq[0] == '[' && seq[1] == 'B')
{
    if (history_position < history_count - 1)
    {
        history_position++;
        display_history_command(
            buffer,
            &length,
            history_position
        );
    }
    else
    {
        clear_line(length);
        length = 0;
        buffer[0] = '\0';
        history_position = history_count;
    }

    continue;
}

continue;
}

/* Normal character */

```

```

        if (length < BUFFER_SIZE - 1)
        {
            buffer[length] = c;
            length++;

            write(STDOUT_FILENO, &c, 1);
        }
    }
}

disable_raw_mode(&original);

return 0;

}

```

Output-

SKILL-03 TASK-01: COMMAND HISTORY

=====

Compilation:

```
gcc -Wall -Wextra command_history.c -o command_history
```

Execution:

```
./command_history
```

Sample Output:

Simple Command History

Type commands. Use Up/Down arrows to navigate.

Press Ctrl+D or type exit to quit.

```
shell> hello
```

Command stored: hello

shell> linux

Command stored: linux

shell> operating systems

Command stored: operating systems

HISTORY TEST

=====

After entering the commands:

hello

linux

operating systems

Pressing the UP arrow recalls:

operating systems

Pressing UP again recalls:

linux

Pressing UP again recalls:

hello

Pressing DOWN moves forward through the command history.

The program uses escape sequences generated by the terminal to identify UP and DOWN arrow keys.

UP arrow escape sequence:

ESC [A

DOWN arrow escape sequence:

ESC [B

Backspace is handled by removing the last character from the input buffer and moving the cursor back.

The input buffer is updated whenever a character is entered, deleted, or a history command is recalled.

Skill-03 Task-02: Dynamic Memory Management

Objective

To dynamically allocate buffers, resize arrays, prevent buffer overflow, manage linked lists, release memory correctly, and verify memory usage using Valgrind.

Features

- Dynamic memory allocation using `malloc()`
- Dynamic array resizing using `realloc()`
- Bounds checking before inserting data
- Linked-list creation
- Dynamic node allocation

- Proper memory deallocation using free()
- Valgrind memory verification

Dynamic Array

The program initially allocates memory for two integers.

```
int *array = malloc(size * sizeof(int));
```

```
#include <stdio.h> #include <stdlib.h> #include <string.h>
```

```
typedef struct Node { int value; struct Node *next; } Node;
```

```
void demonstrate_dynamic_array() { int size = 2; int count = 0;
```

```
int *array = malloc(size * sizeof(int));
```

```
if (array == NULL)
{
    perror("malloc");
    exit(EXIT_FAILURE);
}
```

```
printf("Dynamic Array:\n");
```

```
for (int i = 1; i <= 5; i++)
{
```

```
    if (count == size)
    {
        size *= 2;
```

```
        int *temp = realloc(array, size * sizeof(int));
```

```
        if (temp == NULL)
        {
            free(array);
            perror("realloc");
        }
    }
}
```

```

        exit(EXIT_FAILURE);
    }

    array = temp;

    printf("Buffer resized to %d elements\n", size);
}

array[count] = i * 10;

printf("Added: %d\n", array[count]);

count++;
}

printf("Array contents: ");

for (int i = 0; i < count; i++)
    printf("%d ", array[i]);

printf("\n");

free(array);

}

void demonstrate_linked_list() { Node *head = NULL;

printf("\nLinked List:\n");

for (int i = 1; i <= 3; i++)
{
    Node *new_node = malloc(sizeof(Node));

    if (new_node == NULL)
    {
        perror("malloc");
        exit(EXIT_FAILURE);
    }

    new_node->value = i * 100;
    new_node->next = head;

```

```

        head = new_node;

        printf("Added node: %d\n", new_node->value);
    }

    printf("List contents: ");

    Node *current = head;

    while (current != NULL)
    {
        printf("%d ", current->value);
        current = current->next;
    }

    printf("\n");

    /* Release linked-list memory */
    current = head;

    while (current != NULL)
    {
        Node *temp = current;
        current = current->next;
        free(temp);
    }

    printf("Linked list memory released successfully.\n");

}

int main(){ printf("Dynamic Memory Management Demonstration\n");
printf("=====\n\n");

demonstrate_dynamic_array();

demonstrate_linked_list();

printf("\nAll dynamically allocated memory was released.\n");

```

```
return 0;
```

```
}
```

Output-

SKILL-03 TASK-02: DYNAMIC MEMORY MANAGEMENT

=====

Compilation:

```
gcc -Wall -Wextra -g dynamic_memory.c -o dynamic_memory
```

Execution:

```
./dynamic_memory
```

Output:

Dynamic Memory Management Demonstration

=====

Dynamic Array:

Added: 10

Added: 20

Buffer resized to 4 elements

Added: 30

Added: 40

Buffer resized to 8 elements

Added: 50

Array contents: 10 20 30 40 50

Linked List:

Added node: 100

Added node: 200

Added node: 300

List contents: 300 200 100

Linked list memory released successfully.

All dynamically allocated memory was released.

VALGRIND VERIFICATION

=====

Command:

valgrind --leak-check=full ./dynamic_memory

Important result:

All heap blocks were freed -- no leaks are possible.

ERROR SUMMARY: 0 errors from 0 contexts

OBSERVATION

=====

The program dynamically allocates memory using malloc().

`realloc()` is used to resize the dynamic array when the allocated capacity becomes full.

The array size is increased when more elements are required.

A linked list is created using dynamically allocated nodes.

Each node is released using `free()` after use.

Valgrind is used to verify that dynamically allocated memory has been released correctly.